

# Module 10 – Kubernetes Security

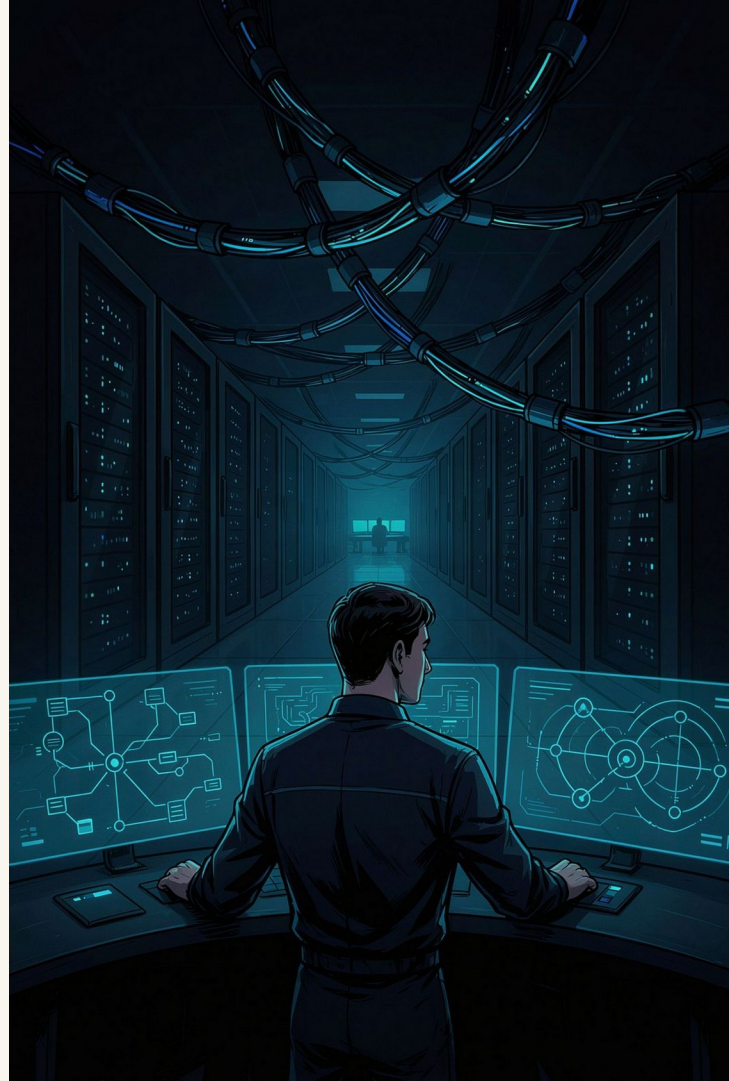
Network Policies, Pod Security Standards, and CIS Kubernetes Benchmark

A production-grade security deep dive for EKS Anywhere administrators. This module covers the three pillars of Kubernetes cluster hardening: controlling traffic flow with Network Policies, enforcing workload security posture with Pod Security Standards, and validating compliance with the CIS Kubernetes Benchmark.

EKS ANYWHERE TRAINING

MODULE 10

PRODUCTION SECURITY



# Kubernetes Security Hardening Overview

Securing a Kubernetes cluster requires a layered, systematic approach — not a single control. Production hardening targets every plane: the API server, the data plane, the workload runtime, and the supply chain. Each layer closes a different attack vector.

## Control Plane

API Server TLS, RBAC, audit logging, etcd encryption at rest, kubeconfig hardening.

## Data Plane

Network Policies, CNI security, node hardening, kubelet authentication, CIS benchmarks.

## Workload Runtime

Pod Security Standards, seccomp, AppArmor, read-only root FS, non-root execution.

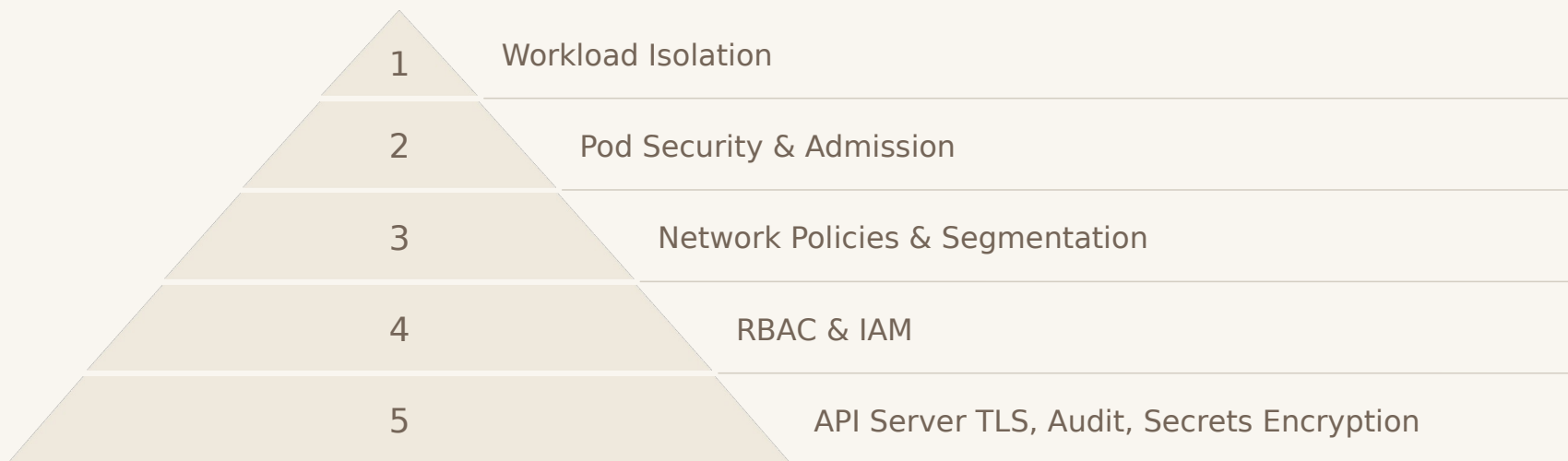
## Supply Chain

Image signing (Cosign/Notation), vulnerability scanning, admission webhooks, Kyverno policies.

📌 EKS Anywhere inherits AWS security best practices but demands explicit hardening — defaults are never sufficient for production.

# Defense in Depth – Security Layers in Kubernetes

Defense in Depth means no single control is trusted to stop all threats. Each layer independently enforces policy so that a breach at one layer does not cascade into full cluster compromise.



Each tier assumes the tier below may be compromised. This philosophy directly aligns with the NIST Zero Trust Architecture (SP 800-207) and is required for DoD IL4/IL5 EKS Anywhere deployments.

# Default Kubernetes Networking Model and Security Risks

## The Default: Flat, Open Networking

Out of the box, Kubernetes implements a fully flat network. Every Pod can reach every other Pod across all namespaces — no firewall, no segmentation, no ACL. This is by design for developer convenience, but catastrophic in production.

- Any compromised Pod can probe the entire cluster network
- Lateral movement is trivial — no east-west controls
- Namespace boundaries are **logical only**, not network-enforced
- Services are reachable cluster-wide unless explicitly restricted
- etcd, the kubelet API, and metadata endpoints can be contacted from workload Pods

## Key Risk Vectors

**SSRF attacks** can pivot from one microservice to the entire cluster network including cloud metadata APIs.

**Compromised Pods** performing internal port scans go undetected without egress controls.

**Supply chain compromise** gains unrestricted internal access without Network Policies in place.



Never run production workloads on a cluster with zero Network Policies. A single compromised container has full east-west network access.

# Network Policies – Purpose, Benefits, and Zero Trust Networking

Kubernetes Network Policies are the native mechanism to implement Zero Trust networking at the Pod level. They define explicit allow rules for ingress and egress traffic — everything not explicitly permitted is denied.



## Micro-Segmentation

Isolate workloads by namespace, label, or IP block. Limit blast radius from any single compromise to its immediate network neighborhood.



## Zero Trust Enforcement

Enforce explicit-deny by default. Traffic must be explicitly permitted by policy — implicit trust between Pods is eliminated.



## Compliance Alignment

Satisfies PCI-DSS network segmentation requirements, NIST 800-204 microservices controls, and CIS Benchmark network checks.



## Lateral Movement Prevention

Block east-west propagation of threats. A breach in the frontend tier cannot directly reach database Pods without an explicit policy exception.

# Network Policy Architecture

Frontend → Backend → Database Communication Model

1

## Frontend Tier

Accepts ingress from the Ingress Controller only. May reach Backend on port 8080. **Cannot** reach Database directly.

2

## Backend Tier

Accepts ingress from Frontend Pods only. May reach Database on port 5432. **Cannot** reach external internet without explicit egress policy.

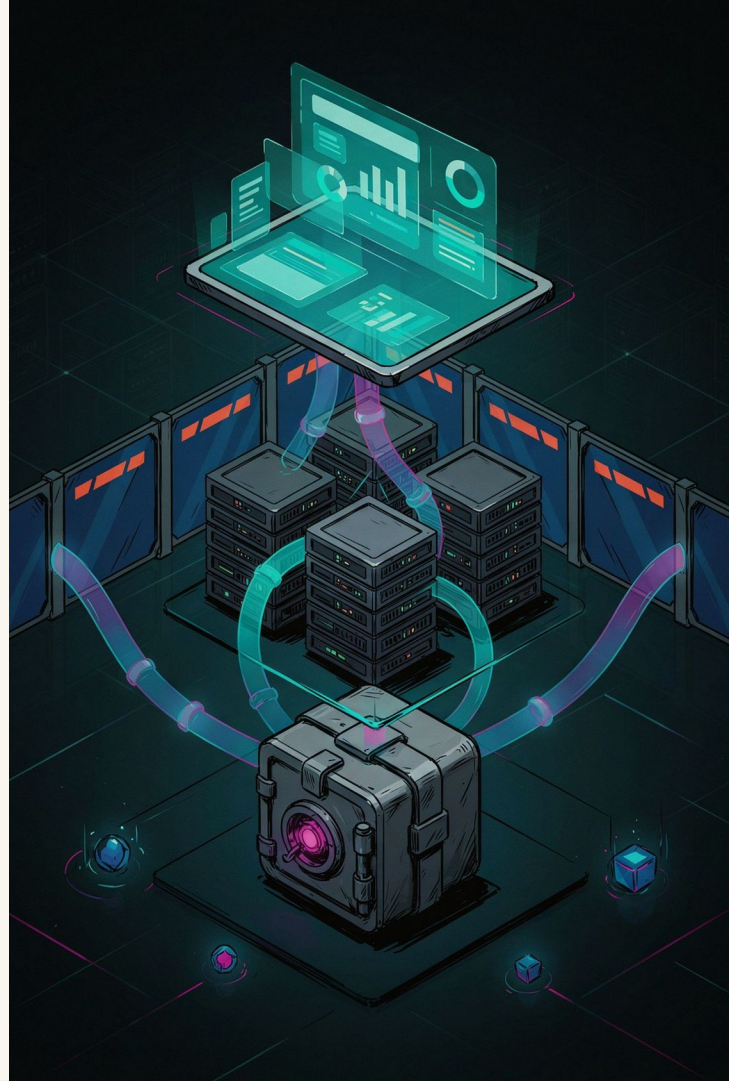
3

## Database Tier

Accepts ingress **only** from Backend Pods. No egress permitted except DNS (UDP 53). Zero external exposure.



This three-tier model is the reference architecture for production EKS Anywhere deployments. Each tier lives in its own namespace with a default-deny policy applied first.



# Network Policy Components

Understanding the selector model is critical to writing policies that behave correctly. Each field narrows or expands the scope of allowed traffic independently.

## Selectors & Scope

### podSelector

Selects the target Pods within the policy's namespace using label selectors. An empty `{}` matches all Pods in the namespace.

### namespaceSelector

Matches source/destination namespaces by label. Required for cross-namespace traffic. Combine with `podSelector` using AND logic for precision.

### ipBlock

Matches CIDR ranges for traffic to/from external IPs. Use `except` to exclude sub-ranges (e.g., exclude the node CIDR from a broad allow).

## Policy Directionality

### ingress

Defines allowed inbound traffic to selected Pods. Each rule specifies `from` sources and `ports`. Multiple rules are OR'd together.

### egress

Defines allowed outbound traffic from selected Pods. Critical for preventing data exfiltration and SSRF-based cloud metadata abuse.

### policyTypes

Explicitly declares `Ingress`, `Egress`, or both. If `Egress` is declared but no egress rules exist, all outbound is blocked — including DNS.

# Real-World Network Policy Examples

Production Network Policies follow a layered application order: default-deny first, then explicit allows. These four patterns cover the majority of enterprise segmentation requirements.



## Default Deny All

Apply to every namespace first. `podSelector: {}` with `policyTypes: [Ingress, Egress]` and no rules. Blocks all traffic until explicit allows are added. The non-negotiable baseline.



## Namespace Isolation

Allow intra-namespace traffic only using `namespaceSelector` matching the current namespace label. Prevents cross-namespace lateral movement without explicit exceptions.



## Database Protection

Restrict ingress to database Pods to only backend Pods identified by label selector. Port-specific allow on 5432/3306. No other traffic reaches the data tier under any circumstance.



## Egress Control

Allow DNS egress (UDP/TCP 53 to kube-dns), specific internal service CIDRs, and external API endpoints. Block all other outbound to prevent exfiltration and metadata API abuse.



Always apply default-deny before adding allow rules. In EKS Anywhere, apply these policies via GitOps (Flux/ArgoCD) so drift is immediately detected and corrected.



# Enterprise Network Policy Best Practices & Micro-Segmentation

## Micro-Segmentation Strategy

Micro-segmentation goes beyond namespace isolation — it enforces policy at the workload identity level using Pod labels. Each microservice gets its own Network Policy profile.

- **Label consistently:** Use `app`, `tier`, and `component` labels as policy anchors — enforce via admission webhooks
- **Namespace per team:** Assign namespaces by team/environment and enforce cross-namespace policies explicitly
- **Least-privilege egress:** Allow only required external endpoints; DNS must be explicitly permitted
- **Version control all policies:** Store in Git, deploy via Flux/ArgoCD, audit with `kube-bench`

## CNI Requirements

Network Policies are only enforced if the CNI plugin supports them. Verify your CNI before relying on any policy.

| Supported CNIs                                                                              | Not Supported                                                                                                                          |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Calico, Cilium, Weave Net, Antrea — full NetworkPolicy v1 support with optional extensions. | Flannel — does not enforce Network Policies. Policies are silently ignored. Replace with Calico or Cilium for EKS Anywhere production. |

# Pod Security Standards – Overview and Threat Model

Pod Security Standards (PSS) replace the deprecated PodSecurityPolicy (PSP) admission plugin. Introduced in Kubernetes 1.22 (stable 1.25), PSS defines three built-in security profiles enforced via the Pod Security Admission controller — a native, webhook-free approach to workload hardening.

1

## Threat: Privileged Containers

Root-equivalent access to the host kernel. A single privileged container can escape to the node, pivot the cluster, and exfiltrate IAM credentials.

2

## Threat: HostPath Mounts

Direct access to node filesystem enables reading `/etc/kubernetes/pki`, kubelet tokens, and cloud provider credentials. Full node compromise path.

3

## Threat: Privilege Escalation

Containers that can call `setuid/setgid` or use `sudo` can elevate from a restricted user to root within the container, bypassing runtime controls.

4

## Threat: Unrestricted Syscalls

Without seccomp profiles, containers can invoke any kernel syscall, enabling kernel exploit chains (Dirty COW, runc vulnerabilities, etc.).

# PSS Level Comparison: Privileged vs. Baseline vs. Restricted

Choose the most restrictive level your workloads can tolerate. Operator-level system components may require Privileged; all application workloads should target Restricted.

| Security Control      | Privileged                     | Baseline               | Restricted                           |
|-----------------------|--------------------------------|------------------------|--------------------------------------|
| Privileged Containers | Allowed                        | Blocked                | Blocked                              |
| hostNetwork / hostPID | Allowed                        | Blocked                | Blocked                              |
| hostPath Volumes      | Allowed                        | Blocked                | Blocked                              |
| Run as Root           | Allowed                        | Allowed                | Blocked                              |
| Privilege Escalation  | Allowed                        | Allowed                | Blocked                              |
| seccomp Profile       | Any / None                     | Any / None             | RuntimeDefault or Localhost required |
| Linux Capabilities    | Unrestricted                   | Limited dangerous caps | Only NET_BIND_SERVICE allowed        |
| Typical Use Case      | CNI, storage, system operators | General workloads      | All production apps (target)         |

☐ Migrate all application workloads to Restricted. Use Baseline only as a stepping stone during remediation. Privileged is reserved strictly for cluster system components.

# Restricted Policy – Production Deep Dive

The Restricted profile enforces the maximum security posture available natively in Kubernetes. Each control maps directly to a specific container breakout or privilege escalation technique.

## No Privileged & No HostPath

`privileged: false` prevents kernel-level access. Blocking `hostPath` volumes eliminates node filesystem reads of PKI material and cloud credentials.

## Non-Root Execution

`runAsNonRoot: true` + `runAsUser: ≥1000`. Prevents UID 0 exploitation. Combined with `readOnlyRootFilesystem: true` to block runtime writes to container FS.

## seccomp + AppArmor

Set `seccompProfile.type: RuntimeDefault` to block ~300 dangerous syscalls. AppArmor profile `runtime/default` adds MAC-layer filesystem and network access controls.

## Capability Dropping

`capabilities.drop: [ALL]` then selectively re-add only `NET_BIND_SERVICE` if required. Eliminates dangerous capabilities like `SYS_ADMIN`, `NET_RAW`, `SYS_PTRACE`.

## No Privilege Escalation

`allowPrivilegeEscalation: false` blocks `setuid/setgid` binaries from gaining elevated privileges. Critical for preventing in-container root escalation.

# Pod Security Admission – Architecture and Namespace Labels

Pod Security Admission (PSA) is a built-in admission controller (enabled by default since Kubernetes 1.25) that evaluates Pod specifications against PSS profiles at admission time. Configuration is namespace-scoped via labels.



Apply all three modes simultaneously during migration to enforce strict policy while surfacing violations without blocking workloads: `pod-security.kubernetes.io/enforce: restricted | pod-security.kubernetes.io/warn: restricted | pod-security.kubernetes.io/audit: restricted`

## Migration Strategy

- Start with `warn + audit` only — collect violations without blocking

## Exempt System Namespaces

Always exempt: `kube-system`, `kube-public`, `kube-node-lease`, and `CNI/storage operator` namespaces. Configure exemptions in

# Pod Security Best Practices for Production Clusters

01

## Enforce Restricted Cluster-Wide by Default

Set the cluster-level default PSA mode to `restricted` via `AdmissionConfiguration`. Individual namespaces can override for system components. New namespaces inherit Restricted automatically.

02

## Pair PSA with Kyverno or OPA Gatekeeper

PSA provides coarse-grained profile enforcement. Kyverno/Gatekeeper enforce fine-grained organizational policies: required labels, image registry restrictions, resource limit requirements, and naming conventions.

03

## Scan Images Before Admission

Integrate Trivy, Gype, or Snyk into the CI pipeline. Reject images with Critical CVEs at build time. Use admission webhooks (Kyverno ImageVerify) to verify image signatures at deploy time.

04

## Apply Security Context at Both Pod and Container Level

Pod-level `securityContext` sets defaults; container-level overrides allow exceptions. Explicit is better than implicit — always declare `runAsNonRoot`, `readOnlyRootFilesystem`, and `allowPrivilegeEscalation: false` explicitly.

# CIS Kubernetes Benchmark

## Purpose, Authority, and Compliance Value

The **Center for Internet Security (CIS) Kubernetes Benchmark** is the industry-standard hardening guide for Kubernetes clusters. Published and maintained by CIS with input from cloud vendors and the CNCF community, it provides prescriptive, scored controls across every cluster component.

- **Scored controls** directly impact compliance posture — failures must be remediated
- **Not Scored controls** are recommended but organizationally discretionary
- Aligns with **NIST SP 800-190** (container security), **SOC 2**, **PCI-DSS**, and **FedRAMP** requirements
- AWS publishes an EKS-specific CIS Benchmark variant covering EKS Anywhere node configurations

## Automated Auditing with kube-bench

**kube-bench** (Aqua Security) automates CIS Benchmark checks against running clusters. Run as a Kubernetes Job or DaemonSet. Outputs PASS/FAIL/WARN per control with remediation commands. Integrate into your CI/CD pipeline for continuous compliance validation.

```
kubectl apply -f kube-bench-job.yaml
```

# CIS Benchmark Categories

The CIS Kubernetes Benchmark organizes controls into nine functional domains. Each maps to a specific cluster component and its attack surface.



## API Server & etcd

TLS everywhere, anonymous auth disabled, authorization mode set to `Node`, RBAC, etcd peer certs, encryption-at-rest for Secrets. Most scored controls live here.



## Scheduler & Controller Manager

Bind to loopback only (`--bind-address=127.0.0.1`), profiling disabled, `--use-service-account-credentials=true` for least-privilege controller identity.



## Kubelet

Webhook authn/authz, anonymous auth disabled, read-only port 10255 closed, certificate rotation enabled, `--protect-kernel-defaults=true`. Each node is an attack surface.



## RBAC, Secrets & Networking

No wildcard RBAC, Secrets encrypted at rest (AES-GCM/KMS), ServiceAccount token automounting disabled by default, NetworkPolicies covering all namespaces, audit logging to immutable storage.



# Enterprise Kubernetes Hardening Checklist

Use this checklist as a go/no-go gate before promoting any EKS Anywhere cluster to production. All items marked critical must pass before workloads are onboarded.

## Access & Policy Controls

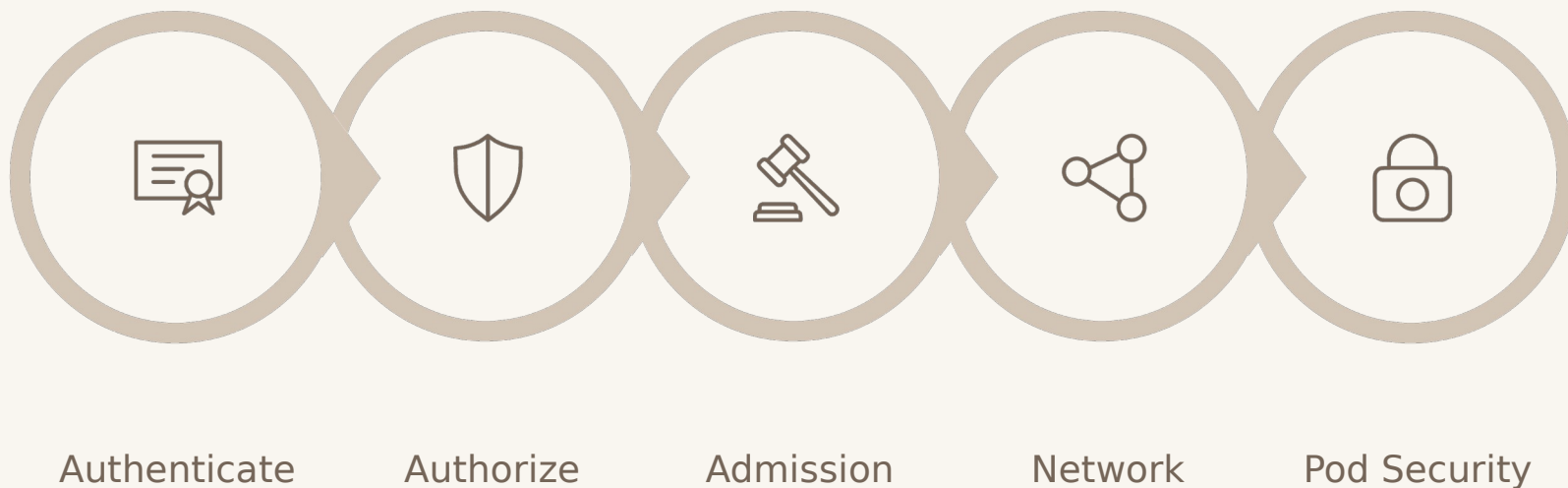
- ☒ RBAC: No cluster-admin service accounts, wildcard verbs audited and remediated
- ☒ Network Policies: Default-deny applied to all namespaces, explicit allows documented
- ☒ Pod Security Admission: Restricted enforced cluster-wide, system namespaces exempted
- ☒ Secrets: Encryption at rest via KMS provider (AWS KMS for EKS Anywhere)
- ☒ TLS: mTLS enforced between control plane components, kubelet certificate rotation enabled
- ☐ Image Signing: Cosign/Notation signatures verified at admission via Kyverno ImageVerify

## Detection, Compliance & Recovery

- ☒ Kyverno / OPA Gatekeeper: Organizational policies deployed and tested in audit mode first
- ☒ Falco: Runtime threat detection deployed, alert pipeline connected to SIEM
- ☒ kube-bench: CIS Benchmark score  $\geq 90\%$  scored controls passing before production promotion
- ☐ Vulnerability Scanning: Trivy/Snyk integrated in CI, critical CVEs block pipeline promotion
- ☐ Audit Logging: API server audit logs forwarded to immutable CloudWatch/S3 destination
- ☐ Disaster Recovery: etcd backup tested, cluster restore RTO validated <4 hours

# Kubernetes Security Workflow – End-to-End

From Initial Request to Hardened Running Workload



Every API request traverses this pipeline sequentially — a failure at any stage rejects the operation. This is Defense in Depth in action: authentication cannot be bypassed by manipulating RBAC, and RBAC bypass cannot circumvent admission control. Each gate is independent.

- ☐ In EKS Anywhere, integrate this workflow with AWS IAM Roles for Service Accounts (IRSA) at the Authentication stage and AWS KMS at the Admission/Secrets stage for a cloud-native Zero Trust posture.

# Key Takeaways — Module 10

## Default Deny First

Always apply default-deny Network Policies before adding allows. Never trust the flat Kubernetes network model in production. Every namespace needs a baseline deny policy.

## Target Restricted PSS

All application workloads must target the Restricted Pod Security Standard. Migrate iteratively using warn/audit modes. Non-root, read-only FS, no privilege escalation — non-negotiable for production.

## Benchmark Continuously

Run kube-bench on every cluster, every release cycle. Aim for  $\geq 90\%$  scored controls passing. CIS Benchmark compliance is both a security requirement and an audit artifact for SOC 2 / FedRAMP.

## Defense in Depth

Security is a layered system. RBAC + Network Policies + Pod Security + Admission Webhooks + Runtime Detection (Falco) + Image Scanning — each layer compensates for the others' limitations.

# Module 10 – Complete

Kubernetes Security: Network Policies, Pod Security Standards & CIS Benchmark

## What We Covered

01

### Defense in Depth & Security Layers

The security model from API server through workload runtime.

02

### Network Policies & Zero Trust

Default-deny, micro-segmentation, three-tier architecture, CNI requirements.

03

### Pod Security Standards

Privileged → Baseline → Restricted profile migration, PSA namespace labels, production best practices.

04

### CIS Benchmark & Hardening Checklist

kube-bench automation, nine control domains, production go/no-go gate criteria.

## Next Steps

**Lab Exercise:** Apply default-deny Network Policies to the training cluster, migrate the sample application namespace to Restricted PSS, and run kube-bench — achieve ≥80% scored controls passing.

**Module 11 Preview:** Supply Chain Security — image signing with Cosign, SBOM generation, Kyverno policy-as-code, and runtime threat detection with Falco on EKS Anywhere.

4-HOUR LAB FOLLOWS